

1. Turing Machines

Introduction

- **Complexity theory** is basically **the study of what's hard or easy to solve with a computer.**
- Given a problem, can this problem be solved with a computer?
- What kind of computer can solve this problem?
- What computational resources are needed?
- Main resources are:
 - **CPU time**
 - **Memory space**
 - **Communication bandwidth**
- In it, the key thing is how the number of steps it takes to solve a problem grows with the size of the input.
- Complexity theory finds answers to the above questions in a formal way.

- **Computational complexity** attempts to classify computational **problems** based on the amount of **resources** required by **algorithms** to solve them.
- Computational **problems** come in various flavors:
 - a. **Decision problem**

Example: is a number n prime?

(...output is YES/NO)

b. Search problem

A search space: – The set of objects among which we search for the solution

Examples: routes between cities

c. Counting problem

Example: Find the number of cycles in a graph

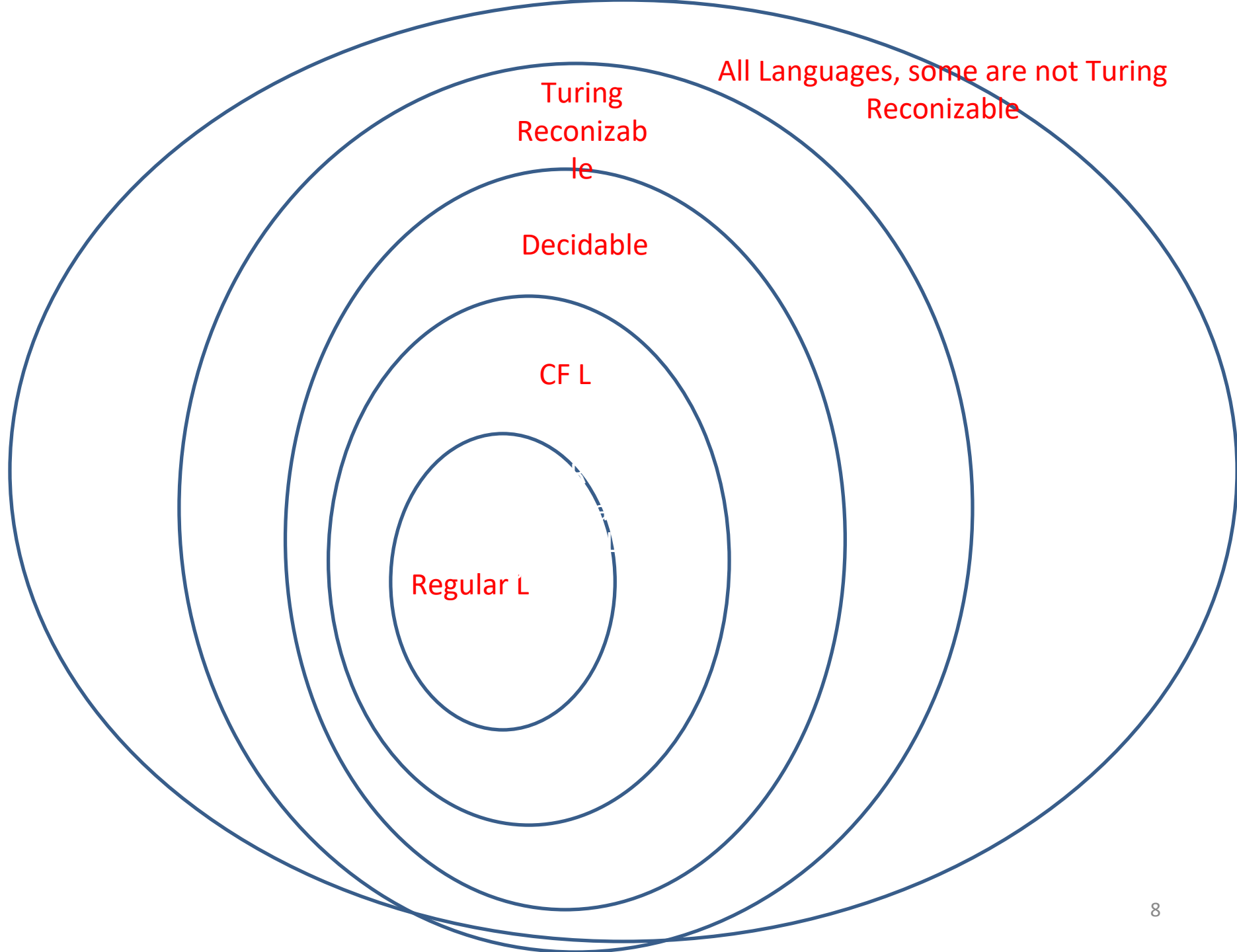
d. Optimization problem

Example: Find a minimum path between two cities

- **Algorithms** are methods of solving problems; they are studied using formal models of computation, like **Turing machines**.

- FAs and PDAs are models of computation associated with **regular languages** and **context-free languages**, respectively
- **Regular languages** form a proper subset of the **context-free languages**
- FAs have limited (a small amount of) memory
- For example, languages, like $L = \{a^n b^n : n \geq 0\}$, are not regular languages
- The use of stack (unlimited LIFO memory) as memory in PDA made it accept the language $L = \{a^n b^n : n \geq 0\}$
- Some simple languages such as $\{a^n b^n c^n : n \geq 0\}$ and $\{ww\}$ are not context-free
- This makes us look beyond context-free languages

- Another classes of Languages:
 - Decidable Languages
 - Turing Recognizable Languages
 - Languages that are not Turing Reconizable



Turing
Reconizable

All Languages, some are not Turing
Reconizable

Decidable

CF L

Regular L

- Data Structures:

FSA- input string

0	1	0	1	1
---	---	---	---	---

-PDA

input string

Stack

C
A
\$

Turing Machine – Tape

1	0	1	1	
---	---	---	---	--

Turing Machines

- Turing machine (TM) is a powerful model of computation first proposed by **Alan Turing** in 1936
- Similar to an FA but with an **unlimited** and **unrestricted memory**
- A TM is an accurate **model of a general purpose computer**
- A TM can do everything a real computer can do
- A TM cannot solve some problems that are beyond the theoretical limits of computation
- A TM model uses an **infinite tape** as its **unlimited memory**
- It has a **tape head** that can **read** and **write** symbols and **move** around on the tape
- Initially the tape contains only the input string and is blank everywhere else

- Turing machines...

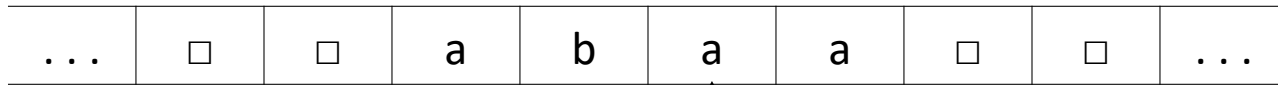
- a memory with head (like a RAM)
- a finite control (like a processor)

- TM memory: infinite tape

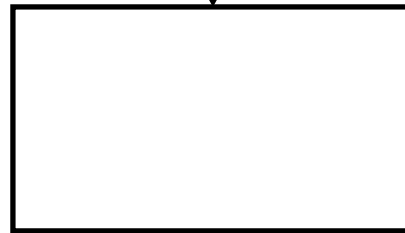
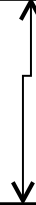
TM I/O: a tape-head that can read/write symbols and move around on the tape

- TM Processor: a control device that performs transformations of the symbols written on the tape while moving the head around.

tape



read-write head



control unit

Comments:

1. What are the similarities to a real computer?
2. What are the differences from a real computer?

Figure 1.1: Schematic representation of a Turing machine

Informal description of Turing machine

What's different from a DFA

- Input tape is infinite
- Input head can both READ and WRITE
- Input head can move LEFT and RIGHT
- Has both ACCEPT and REJECT states and accepts/rejects right away (does not need to reach end of input)

- A TM **recognizes** a language if it accepts all and only those strings in the language
- A TM **decides a language** if it **accepts** all strings in the language and **rejects** all strings not in the language
- A **language** is called **Turing-recognizable** or **recursively enumerable** if some TM recognizes it
- A language is called **decidable** or **recursive** if some TM **decides** it

Initial condition

- Initially the tape contains only the input string and is blank everywhere else
- If TM needs to store information, it may write it on the tape
- To read the information that it has written, TM can move its head back over its tape

The output

- The computation performed by a TM ends up with an output, which is one of:
accept, reject, or compute forever.

- **TM versus FA**

1. A TM can both write on the tape and read from it;
a FA can only read its input
2. The read/write head of a TM can move both to the left and to the right;
a FA can move in one direction only
3. The tape of a TM is infinite;
the input of a FA is finite
4. The special states of a TM for rejecting and accepting the input take immediate effect;
FA terminates when input is entirely

Example:

- Construct a TM M_1 that tests the membership in the language $L = \{ w\#w \mid w \in \{0,1\}^* \}$
- In other words: we want to design M_1 such that $M_1(w) = \text{accept}$, if $w \in L$

Note: with regard to a real computer this problem becomes:

construct a program that solve the above problem

Hence, a TM is an algorithm (i.e., a program).

Strategy

- Identify first the character # in w
- Zig-zag around # to determine whether or not the corresponding places on the two sides of # match
- We can mark the places we have already visited

M1 works following the strategy specified above:

- M1 makes multiple passes over the input with the read/write head
- On each pass M1 matches one of the characters on each side of # symbol
- To keep track of which symbols have been checked M1 crosses off each symbol as it is examined
- If M1 crosses all symbols it accepts, otherwise it rejects.

M1 on input w

1. Scan the input tape to be sure that it contains a single #. If not, reject
2. Zig-zag across the tape to corresponding positions on either side of # to check whether these positions contain the same symbol. If they do not, reject. Cross off the symbols as they are checked
3. When all symbols to the left of # have been crossed off, check for the remaining symbols to the right of #. If any symbol remain, reject; otherwise accept."

Illustration, Fig 2



Figure 2: Snapshots of M_1 computing

Preparing a formal definition

- The control device (the processor) of a TM can be in a finite set of states. We denote this set by Q
- The tape (i.e. the memory) of a TM is split into an infinite number of locations called **squares** or **cells**. Each square can hold a symbol of a given alphabet, Γ .
- The tape-head can move to the right (R) or to the left (L) one square at each step of the computation performed by the TM.

Transitions

- The heart of a formal definition of a TM is the transition function δ because it tells how the machine is going from one step to the next.
- The signature of
 $\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$
- In other words: when TM is in a state $q \in Q$ and the head is over a tape square containing a symbol $a \in \Gamma$
- If $\delta(q,a)=(r,b,L)$ the machine replaces a with b , moves to the state r and moves the head to the left (L)

- If $\delta(q,a)=(r,b,R)$ the machine replaces a with b , moves to the state r and moves the head to the Right (R)

Formal Definition :The Standard Turing Machine

- A Turing machine is a 7 – tuple $(Q, \Sigma, \Gamma, \delta, q_0, \square, F)$ where Q, Σ, Γ are all finite sets and:
 1. Q is the set of states
 2. Σ is the input alphabet $\subseteq \Gamma - \{\square\}$ ($\square$ is not in Σ)
 3. Γ is the **tape alphabet** ($\Sigma \subset \Gamma$)
 4. $\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ is the transition function
 5. $\square \in \Gamma$ is a special symbol called the **blank**
 6. $q_0 \in Q$ is the initial state
 7. $F \subseteq Q$ is the set of final states

Computations:

$M=(Q, \Sigma, \Gamma, \delta, q_0, \square, F)$ computes as follows:

- It receives as input as input $w=w_1w_2... \in \Sigma^*$ written on the leftmost squares of the tape; the rest of the tape is blank (i.e., filled with $\square$)
- The head starts on the leftmost square of the tape
- The first blank encountered shows the end of the input
- Once it starts, it proceeds by the rules describing δ
- If M ever tries to move to the left of the leftmost square the head stays in the leftmost square even though δ indicates L
- Computation continues until M enters **qacc**, **qreject** at which points it halts. If neither occurs M goes on forever

- The heart of a TM is the transition function δ
- When the machine is in a certain state q and the head is over a tape square containing a , and if $\delta(q, a) = (r, b, L)$, the machine writes the symbol b replacing the, and goes to state r

A move of the TM

A move of the TM is a function of the state of the finite control and

the tape symbol scanned. In one move the TM will

1. Change state
2. Write a tape symbol in the cell scanned.
3. Move the tape head left or right

- The third component is either L or R and indicates whether the head moves to the left or right after writing, respectively
- Changes occur in the current state, the current tape contents, and the current head location

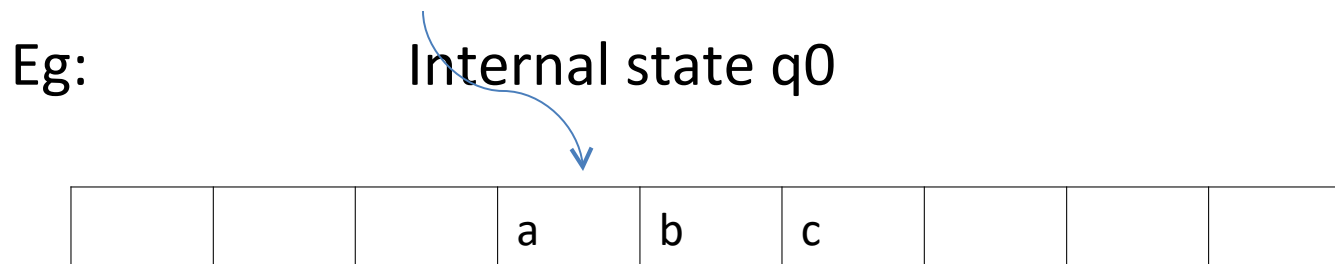


fig a: before the move

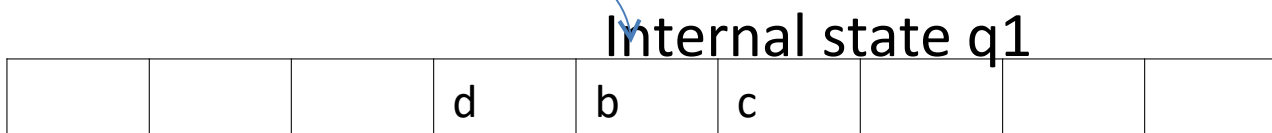


fig b: After the move

$$\delta(q_0, a) = (q_1, d, R).$$

Eg: Consider the Turing machine defined by:

$$Q = \{q_0, q_1\}$$

$$\Sigma = \{a, b\}$$

$$\Gamma = \{a, b, \square\}$$

$$F = \{q_1\}$$

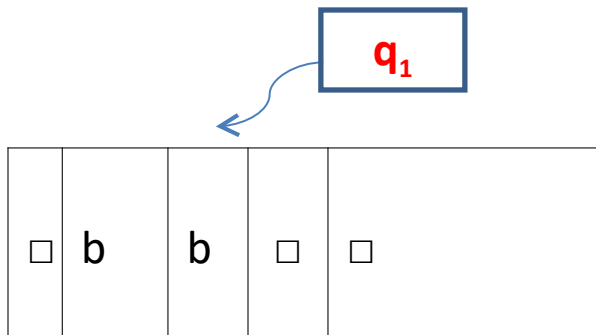
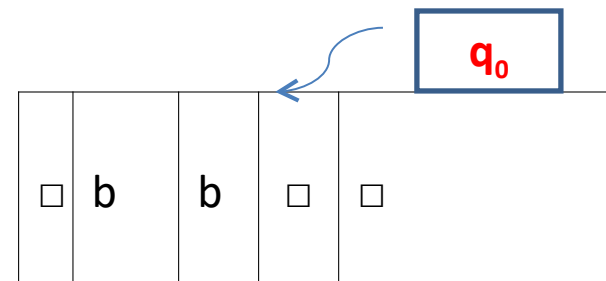
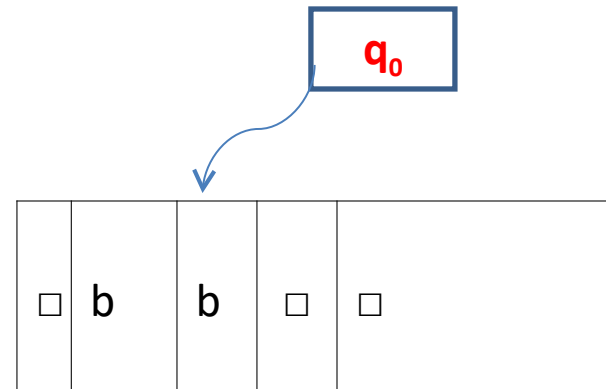
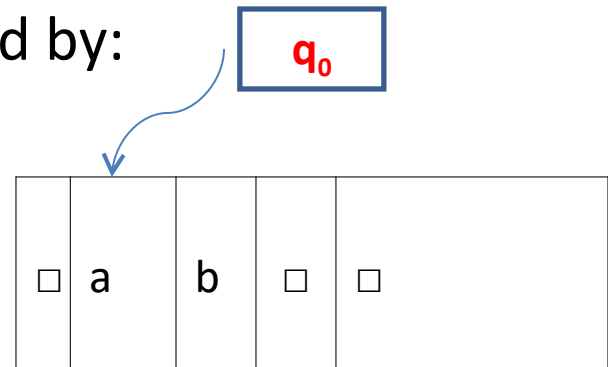
And

$$\delta(q_0, a) = (q_0, b, R)$$

$$\delta(q_0, b) = (q_0, b, R)$$

$$\delta(q_0, \square) = (q_1, \square, L)$$

Show the sequences of moves.



δ can be given using a table.

alphabet symbol	Present State ' q_0 '	Present State ' q_1 '	Present State ' q_2 '
a	1R q_1	1L q_0	1L q_f
b	1L q_2	1R q_1	1R q_f

- Here the transition 1R q_1 implies that the write symbol is 1, the tape moves right, and the next state is q_1 . Similarly, the transition 1L q_2 implies that the write symbol is 1, the tape moves left, and the next state is q_2 .

Transition Diagram

- **Transition graphs** can be used to represent Turing Machines
- The **edges of the graph** can be labeled with 3 items:
 - The **current tape** symbol
 - The **symbol that replaces the current tape** symbol
 - The **direction** in which the read-write head is to move

Example: For the TM shown below, construct its Transition diagram.

$Q=\{q_0,q_1\}$

$\Sigma=\{a,b\}$

$\Gamma=\{a,b, \square\}$

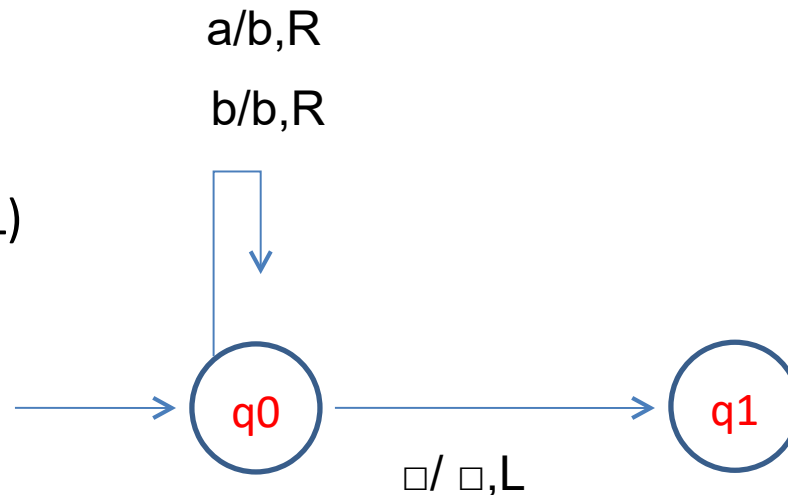
$F=\{q_1\}$

And

$\delta(q_0,a)=(q_0,b,R)$

$\delta(q_0,b)=(q_0,b,R)$

$\delta(q_0, \square)=(q_1, \square,L)$



Configuration of a Turing Machine

Any configuration of a TM is completely determined by :

1. the **current state** of the control unit,
 2. the **contents of the tape**, and
 3. the **position** of the read-write head.
- A setting of these three items is called a **configuration of TM**
 - A configuration of is a tuple

$C = (q \in Q, \text{tapeContent}, \text{HeadLocation})$

- For $q \in Q$, $u, v \in \Gamma^*$, **uqv** also denoted **$C = (u, q, v)$** represents the configuration where **current state is q** , tape contains **uv** and head is on the first symbol of **v**

Notation $C = uqv$

- **Instantaneous description** is a convenient way to exhibit a sequence of configurations
- We use the notation $x_1 q x_2$ or $a_1 \dots a_{k-1} q a_k a_{k+1} \dots a_n$ where the read-write head is on the cell holding a_k , the current state is q , $x_1 = a_1 \dots a_{k-1}$ and $x_2 = a_k a_{k+1} \dots a_n$
- Let $M = (Q, \Sigma, \Gamma, \delta, q_0, \square, F)$ be a TM. Then any string $a_1 \dots a_{k-1} q_1 a_k a_{k+1} \dots a_n$, with $a_i \in \Gamma$ and $q_1 \in Q$, is an **instantaneous description of M**
- A move $a_1 \dots a_{k-1} q_1 a_k a_{k+1} \dots a_n \vdash a_1 \dots a_{k-1} b q_2 a_{k+1} \dots a_n$ is possible iff $\delta(q_1, a_k) = (q_2, b, R)$

- A move $a_1 \dots a_{k-1} q_1 a_k a_{k+1} \dots a_n \vdash a_1 \dots q_2 a_{k-1} b a_{k+1} \dots a_n$ is possible iff

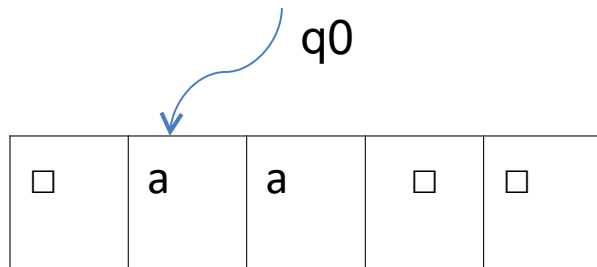
$$\delta(q_1, a_k) = (q_2, b, L)$$
- M is said to **halt** starting from initial configuration $x_1 q_i x_2$ if

$$x_1 q_i x_2 \vdash^* y_1 q_j a y_2$$
for any q_j and a , for which $\delta(q_j, a)$ is **undefined**
- The **sequence of configurations** leading M to a **halt state** is called a **computation**
- If M goes into a loop and never halts we use a special notation

$$x_1 q_i x_2 \vdash^*$$

Eg: Consider the Turing machine defined by:

$Q=\{q_0, q_1\}$ $\Sigma=\{a, b\}$ $\Gamma=\{a, b, \square\}$ $F=\{q_1\}$	And $\delta(q_0, a) = (q_0, b, R)$ $\delta(q_0, b) = (q_0, b, R)$ $\delta(q_0, \square) = (q_1, \square, L)$
Show the sequence of	



Solution:

$q_0aa \mid - bq_0a \mid - bbq_0\square \mid - bq_1b$

Turing Machines as Language Accepters

- A string w is written on the tape, with blanks filling out the unused portions
- The machine started in the initial state q_0 with the read-write head position on the leftmost symbol of w
- If, after a sequence of moves, the Turing machine enters a final state and halts, then w is considered to be accepted

The language of a TM

Let $M = (Q, \Sigma, \Gamma, \delta, q_0, \square, F)$ be a TM. Then $L(M)$ is the set of strings $w \in \Sigma^*$ such that

$$q_0 w * \vdash \alpha p \beta$$

for some state $p \in F$ and any tape string α and β .

The language of a TM

- The set of languages we can accept using a TM is often called the **recursively enumerable languages** or RE languages.

Recursive languages: Languages for which TM do halt, regardless of whether or not they accept.

TM that always halt, regardless of whether or not they accept, are good model of an algorithm

- If an algorithm to solve a given problem exists, then we say the problem is **decidable**.

Let $M = (Q, \Sigma, \Gamma, \delta, q_0, \square, F)$ be a TM

- Then the language accepted by M is:

$$L(M) = \{w \in \Sigma^+ : q_0 w \vdash^* x_1 q_f x_2 \text{ for some } q_f \in F, x_1, x_2 \in \Gamma^*\}$$

Designing a Turing Machine

- Construct a Turing machine which accepts the language **{aba}** over $\Sigma = \{a, b\}$.

Solution:

- We will assume that on input tape the string 'aba' is placed like this:



q0

$\delta(q_0, a) = \delta(q_1, A, R)$

$\delta(q_1, b) = \delta(q_2, B, R)$

$\delta(q_2, a) = \delta(q_3, A, R)$

$\delta(q_3, \square) = (q_4, \square, R)$

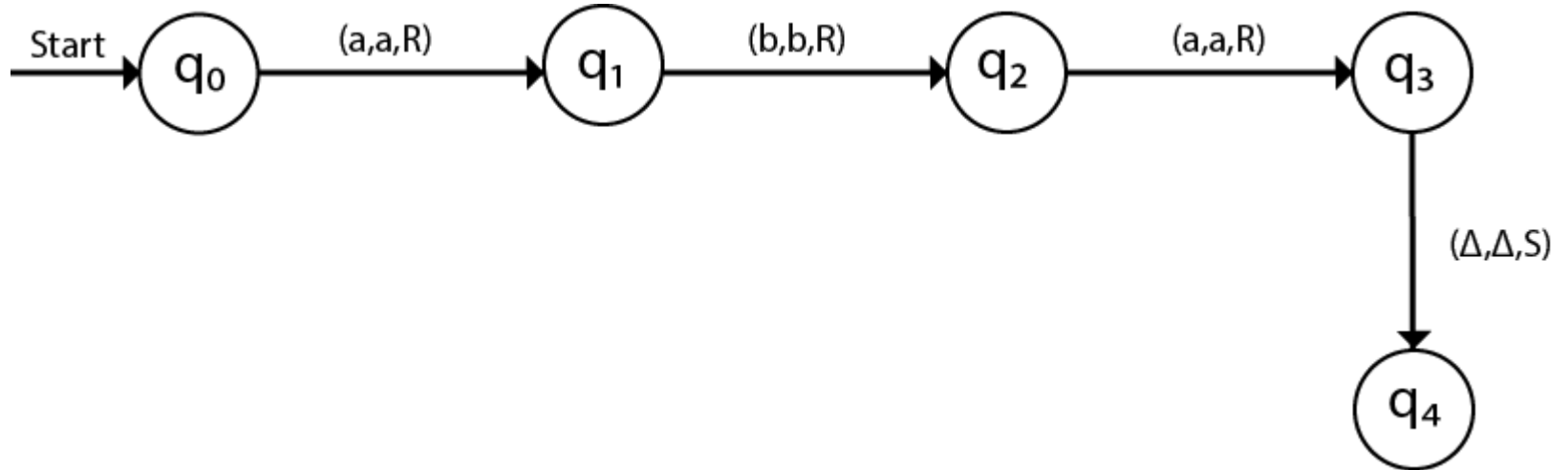
and FINAL state

q4 which is the HALT state

The same TM can be represented by Transition Table:

States	a	b	$\square$
q0	(A, R, q1)	—	—
q1	—	(B, R, q2)	—
q2	(A, R, q3)	—	—
q3	—	—	($\square$, q4, R)
q4	—	—	—

- The same TM can be represented by Transition Diagram
- The same TM can be represented by Transition Diagram



Designing a Turing Machine

2. Design a TM to recognize all strings consisting of an odd number of a's

Solution

- The Turing machine **M** can be constructed by the following moves –
 - Let q_0 be the initial state.
 - If **M** is in q_0 ; on scanning a, it enters the state q_1 and writes $\square$ (blank).
 - If **M** is in q_1 ; on scanning a, it enters the state q_0 and writes $\square$ (blank).
 - From the above moves, we can see that **M** enters the state q_0 if it scans an even number of a's, and it enters the state q_1 if it scans an odd number of a's. Hence q_1 is the only accepting state.

Hence,

$M = \{\{q_0, q_1\}, \{a\}, \{a, \square\}, \delta, q_0, \square, \{q_1\}\}$, where

δ is given by:

$$\delta(q_0, a) = (q_1, \square, R)$$

$$\delta(q_1, a) = (q_0, \square, R)$$

OR

Tape alphabet symbol

a

Present State 'q0'

$\square$ Rq1

Present State 'q1'

$\square$ Rq0

Example 3:

For $\Sigma = \{0, 1\}$, design a TM that accepts the language denoted by the regular expression 00^*

Example 4:

Construct a TM for the language $L = \{0^n 1^n 2^n\}$ where $n \geq 1$

Example 2:

For $\Sigma = \{a, b\}$, design a TM that accepts $L = \{a, b\}^*\{ab\}$

Example 3:

For $\Sigma = \{a, b\}$, design a TM that accepts $L = \{a^n b^n : n \geq 1\}$

Example 4:

Design a TM that accepts $L = \{a^n b^n c^n : n \geq 1\}$

Example 5:

Design a TM that accepts $L = \{xx : x \in \{a, b\}^*\}$

Example 6:

Design a TM that accepts $L = \{a \text{ raised to } 2 \text{ raised to } n : n \geq 0\}$

Turing Machines as Transducers

- In the above TMs, TM either accepts or rejects a string. That is, TM acts as a language acceptor
- A TM can function as a transducer. That is, a string is given as input to TM, it produces some output.
- We can view a TM as a transducer
- A *transducer* transforms input to output
- The input for a computation is a set of symbols on the tape at the initial time
- At the end of the computation, whatever remains on the tape is the output.
- We can view a TM transducer M as an implementation of a function f defined by $w' = f(w)$ provided that $q_0 w \vdash_M^* q_f w'$ for some final state q_f

- A function f with domain D is said to be **Turing-computable** or just **computable** if there exists some Turing machine $M = (Q, \Sigma, \Gamma, \delta, q_0, \square, F)$ such that $q_0 w \vdash_M^* q_f f(w)$, $q_f \in F$, for all $w \in D$
- All common functions are Turing computable.
- This means , basic operations such addition, subtraction, multiplication and division can be performed on it.
- This means TM is an abstract model for modern computer system.

Example 1:

Design A TM that computes the following function:

$$f(x,y)=x+y.$$

Solution:

A positive integer on a TM can be represented by an equal number of 0s.

For example:

Integer 5 is represented by:00000

Integer 8 is represented by:00000000

In the tape, let the two integers be separated using the symbol, \$.

Let us assume that $x=3$ and $y=5$.

The Input tape of the TM will be:

0	0	0	\$	0	0	0	0	0	□
---	---	---	----	---	---	---	---	---	---

own below:

0	0	0	0	0	0	0	0	□	□
---	---	---	---	---	---	---	---	---	---

TM for addition works as follows:'

- Head reads the 0 of the first number, and reaches the separator symbol, \$. Symbol, and \$ is replaced with 0 and head moves toward the right.
- It continues to move towards right till the second number is passed over and □ is reached.
- At □, it turns left and replaces rightmost 0 with □.
- Now, the tape contains the sum of x and y, and TM halts.
- Conversion of the separator symbol , \$ to 0, helps TM to make the single number on the tape.
- The Conversion of the separator symbol , \$ to 0 removes the additional 0 that was generated from the conversion of \$ to 0.

- The definition of δ for this machine given below:

$$\delta(q_0, 0) = (q_0, 0, R)$$

$$\delta(q_0, \$) = (q_1, 0, R)$$

$$\delta(q_1, 0) = (q_1, 0, R)$$

$$\delta(q_1, \square) = (q_2, \square, L)$$

$$\delta(q_2, 0) = (q_f, \square, R)$$

There for, the TM is: $M = (Q, \Sigma, \Gamma, \delta, q_0, \square, F)$ where:

$Q = \{q_0, q_1, q_2, q_f\}$, $\Sigma = \{0\}$, $\Gamma = \{0, \$, \square\}$, $q_0 = \{q_0\}$ and $F = \{q_f\}$ and δ is as given above.

Exercise : Use Instantance descript to show the computation of $3+5$.

Design a TM that copies a string of 1's

- For this problem, let the given string be 111 as shown below.

Input tape:

□	1	1	1	□	□
---	---	---	---	---	---

Note that we store a □ before the string in the tape.

The output from TM should be as follows:

□	1	1	1	1	1	1	□
---	---	---	---	---	---	---	---

- ⊢ _____ symbol x. The TM replaces rightmost x by 1. It goes to the right end of the string and writes a 1 there. Thus TM has added a 1 for the rightmost 1 in the input string. This process is repeated.

- TM reaches q_1 after replacing all 1's by x's and reading the symbol $\square$ at the end of the input string.
- After replacing x by 1, TM reaches q_2 . TM reaches q_3 at the end of the process and halts.
- If the string is 11 we get 1111.
- If the string is 111 we get 111111.

Definition of δ is given below:

$$\delta(q_0, 1) = (q_0, x, R)$$

$$\delta(q_0, \square) = (q_1, \square, L)$$

$$\delta(q_1, x) = (q_1, 1, R)$$

$$\delta(q_1, \square) = (q_2, 1, L)$$

$$\delta(q_2, 1) = (q_2, 1, L)$$

$$\delta(q_2, x) = (q_1, 1, R)$$

$$\delta(q_1, 1) = (q_1, 1, R)$$

$$\delta(q_2, \square) = (q_3, \square, R)$$

Exercise: Design a TM that computes $m-n$ where m and n are positive integers and $m > n$.

Example:

- Let x and y be two positive integers in unary representation. Construct a TM that will halt in a final state q_f if $x \geq y$, and will halt in a nonfinal state q_n if $x < y$.
- More specifically, the machine is to perform the computation:

$q_0 w(x) 0 w(y) \mid - q_f w(x) 0 w(y)$ if $x \geq y$

$q_0 w(x) 0 w(y) \mid - q_n w(x) 0 w(y)$ if $x < y$

□	1	1	1	0	1	1	□
---	---	---	---	---	---	---	---

$\delta(q_0, 1) = (q_1, x, R)$

$\delta(q_1, 1) = (q_1, 1, R)$

$\delta(q_1, 0) = (q_2, 0, R)$

$\delta(q_2, 1) = (q_3, y, L)$

$\delta(q_3, 0) = (q_3, 0, L)$

$\delta(q_3, 1) = (q_3, 1, L)$

$\delta(q_3, x) = (q_0, x, R)$

$\delta(q_2, y) = (q_2, y, L)$

$\delta(q_3, y) = (q_2, y, L)$

$\delta(q_2, \square) = (q_f, \square, R)$

$\delta(q_0, 0) = (q_4, 0, R)$

$\delta(q_4, \square) = (q_f, \square, R)$

Combining Turing Machines for Complicated Tasks

- Basic operations can be put together to perform complicated tasks

Example: Design a TM that computes the function $f(x, y) = x + y$ if $x \geq y$, $f(x, y) = 0$ if $x < y$

- Figure 1.2 shows how TMs can be combined to compute $f(x, y)$

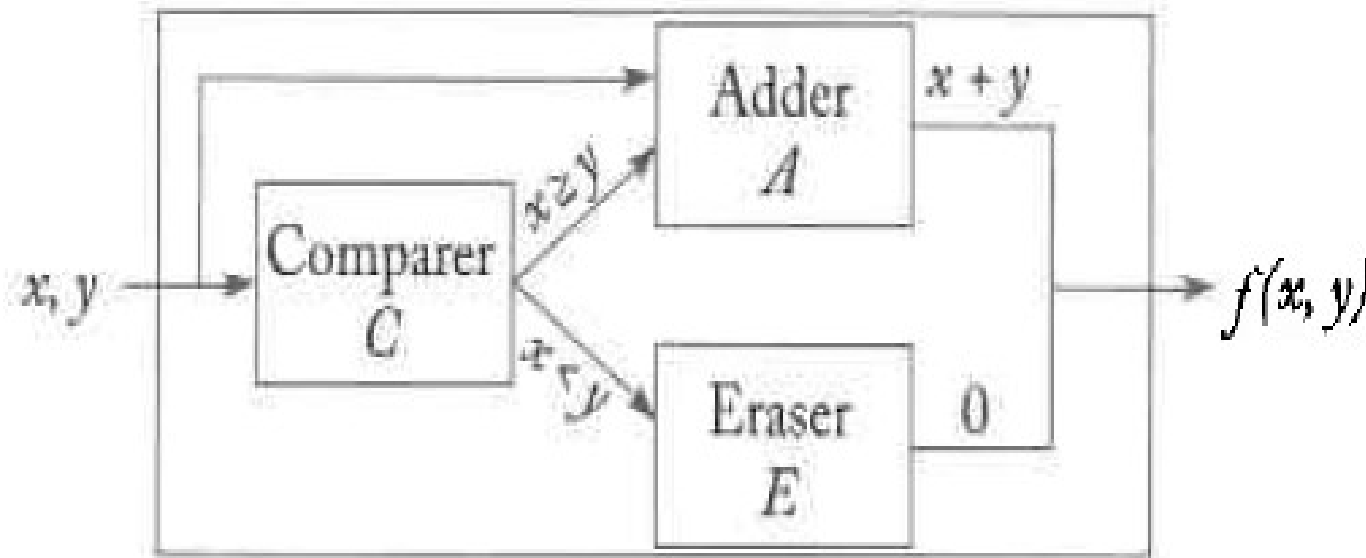


Figure 1.2: A combination of TMs

- The computations to be done by the comparer, C are:

$q_{C,0} w(x)0w(y) \mid^* - q_{A,0} w(x)0w(y)$ if $x \geq y$

$q_{C,0} w(x)0w(y) \mid^* - q_{E,0} w(x)0w(y)$ if $x < y$, where

- $q_{C,0}$ is the initial state of the comparer
- $q_{A,0}$ is the initial state of the adder
- $q_{E,0}$ is the initial state of the eraser

- The computations performed by the adder, A are:

$q_{A,0} w(x)0w(y) \mid^* - q_{A,f} w(x+y)0$ and that of the eraser are:

$q_{E,0} w(x)0w(y) \mid^* - q_{E,f} 0$

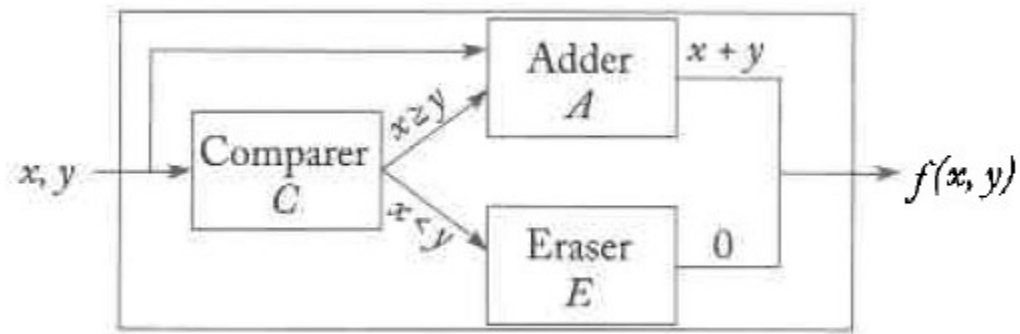


Figure 1.2: A combination of TMs

Turing's Thesis

- Reading Assignment

Other Models of Turing Machines

- The definition of a standard TM is not the only possible one.
- There are alternative definitions that could equally serve well.
- But, the power does not depend on the choice of a TM

Equivalence of Classes of Automata

- Two automata are equivalent if they accept the same language. Consider two classes of automata C_1 and C_2 . If for every automaton M_1 in C_1 there is an automaton M_2 in C_2 such that $L(M_1) = L(M_2)$, we say C_2 is at least as powerful as C_1 . If the converse also holds and for every M_2 in C_2 there is an M_1 in C_1 such that $L(M_1) = L(M_2)$, we say that C_1 and C_2 are equivalent

Turing Machines with a Stay-Option

- In our definition of a standard TM, the read-write head must move either to the right or to the left
- Sometimes it is convenient to provide a third option, to have the read-write head stay in place after rewriting the cell content

The transition function will then be defined as

$\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R, S\}$, where S signifies no movement of the read write head. This option does not extend the power of the power of the automation.

Theorem 1.1: The class of Turing machines with stay-option is equivalent to the class of standard Turing Machines

Proof: Since a TM with stay-option is clearly an extension of the standard model, it is obvious that any standard TM can be simulated by one with a stay-option.

To show the converse, let $M = (Q, \Sigma, \Gamma, \delta, q_0, \square, F)$ be a TM with a stay-option to be simulated by a standard TM $M' = (Q', \Sigma, \Gamma, \delta', q_0', \square, F')$

For each move of M, the simulating machine M' does the following:

- -If the move of M does not involve the stay-option, the simulating machine performs one move, essentially identical to the move to be simulated.
- If S is involved in the move of M, the M' will make two moves:

i.e

$$\delta(q_i, a) = (q_j, b, \text{L or R}),$$

We put into δ' ,

$$\delta'(q_i', a) = (q_j', b, \text{L or R})$$

For each S-transition:

$\delta(q_i, a) = (q_j, b, S)$, we put into δ' the corresponding transitions

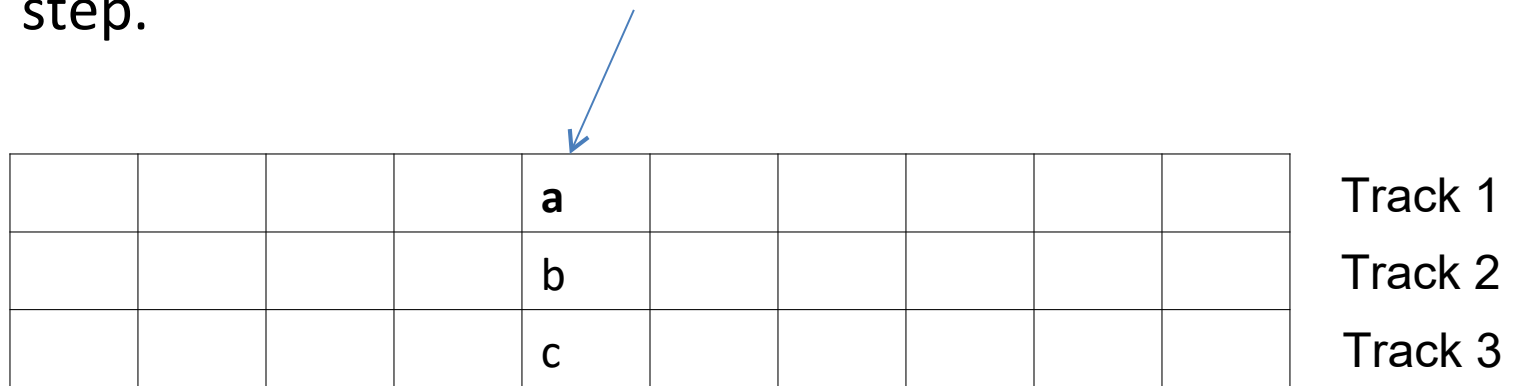
$$\delta'(q_i', a) = (q_j', b, R)$$

And

$$\delta'(q_j', c) = (q_j', c, L) \quad \text{where } c \in \Gamma$$

Multitrack Turing Machines

- A single head and multiple tracks
- In a single move the TM changes all the contents pointed to by the head
- Contain multiple tracks but just one tape head reads and writes on all tracks.
- Here, a single tape head reads n symbols from n tracks at one step.



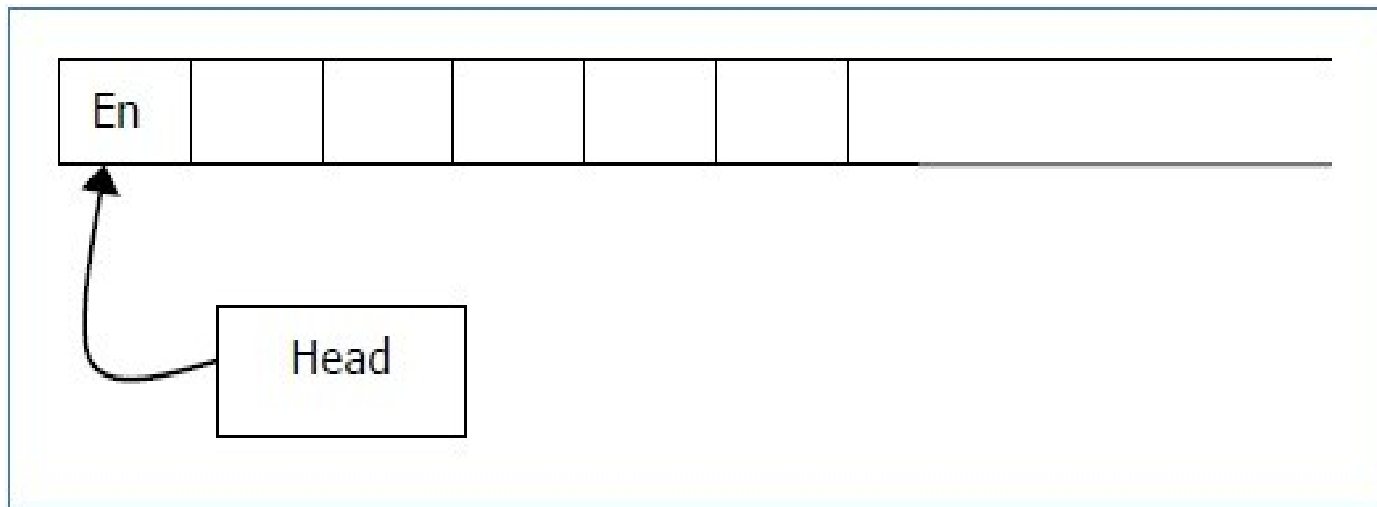
				a						Track 1
				b						Track 2
				c						Track 3

- $\delta(q_i, [a_1, a_2, a_3, \dots]) = (q_j, [b_1, b_2, b_3, \dots], L, R)$
- q_0 is the initial state
- F is the set of final states

Note – For every single-track Turing Machine M , there is an equivalent multi-track Turing Machine M' such that $L(M) = L(M')$

Turing Machines with Semi-Infinite Tape

- A Turing Machine with a semi-infinite tape has a left end but no right end.
- The left end is limited with an end marker.
- This Turing machine is otherwise identical to our standard model except that no left move is permitted when the read-write head is at the boundary



Simulation of standard TM M by Turing Machines with Semi-Infinite Tape

Track 1 for the right part of standard tape

Track 2 for the left part of standard tape

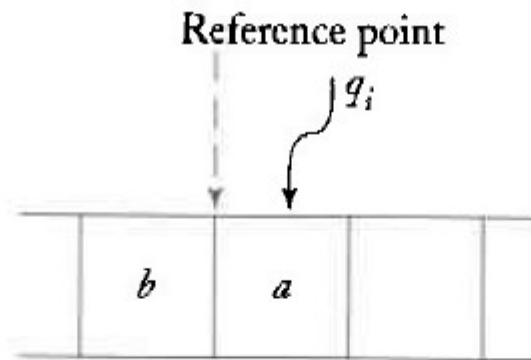
- It is a two-track tape –
 - **Upper track** – It represents the cells to the right of the initial head position.
 - **Lower track** – It represents the cells to the left of the initial head position in reverse order.

- The simulating machine M' has a tape with two tracks.
- On the upper track, we keep information to the right of some reference point on M 's tape.
- The reference point could be the position of the read write head at the start of the computation.
- The lower track contains the left part of the M 's tape in reverse order.
- M' will use information in the upper track only as long as M 's read write head is to the right of the reference point.
- And work on the lower track as M moves into left part of its tape.

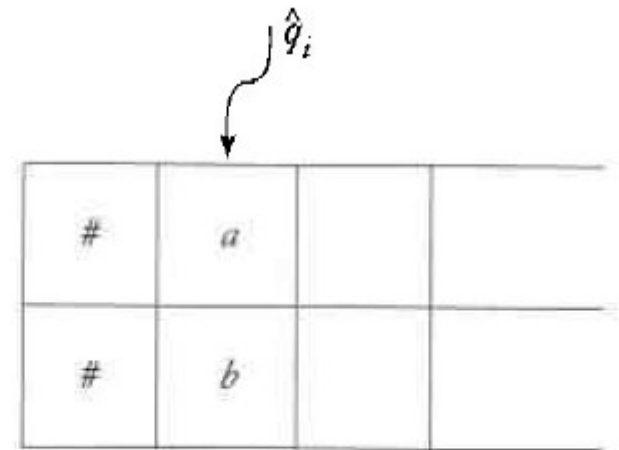
Figure 10.4

(a) Machine to be simulated.

(b) Simulating machine.



(a)



(b)

If M makes the move:

$\delta(q_i, a) = (q_j, c, L)$, the M' makes the moves:

$\delta'(q_i', (a, b)) = (q_j', (c, b), L)$,

Now the simulating machine sees $(\#, \#)$, it then uses the transition:

$\delta'(q_j', (\#, \#)) = (q_j', (\#, \#), R)$,

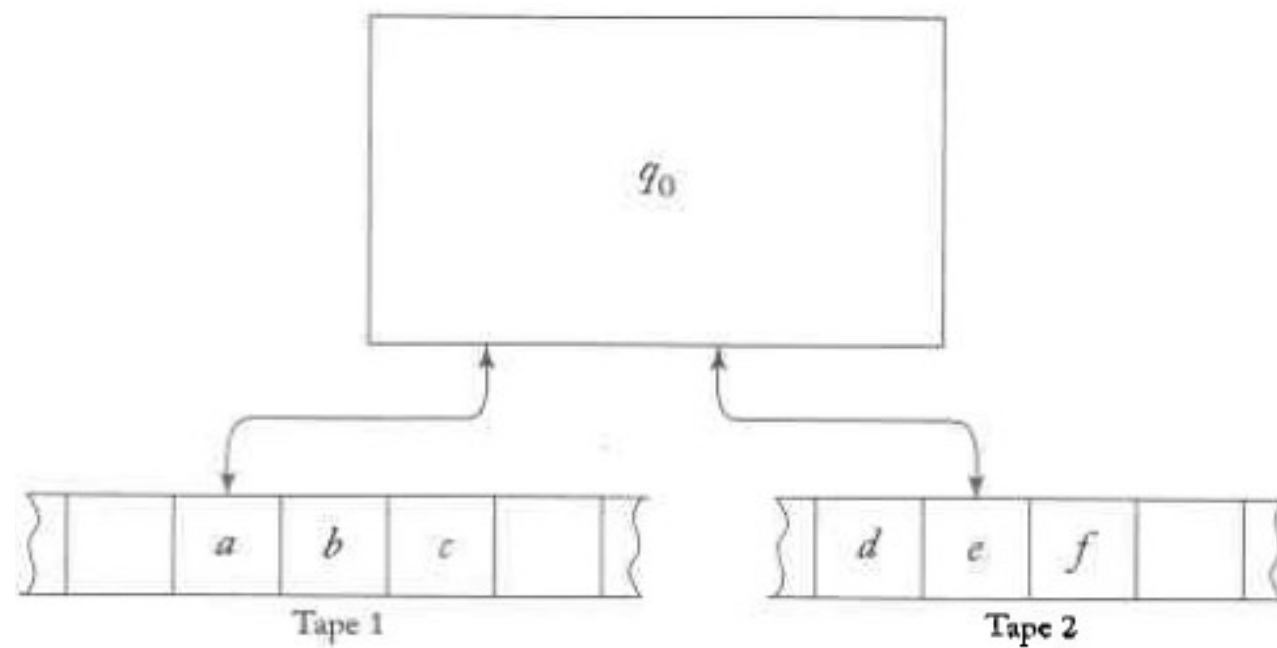
The Off-Line Turing Machine

- Reading Assignment

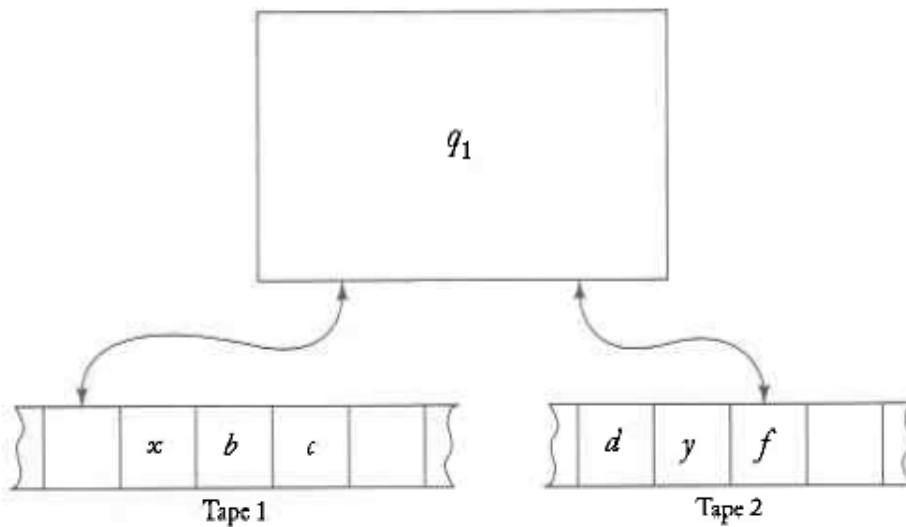
Multitape Turing Machines

- Multi-tape Turing Machines have multiple tapes where **each tape** is accessed with a **separate head**
- Each head can move independently of the other heads.
- The transition function is defined as

$$\delta: Q \times \Gamma^n \rightarrow Q \times \Gamma^n \times \{L, R, S\}^n$$



If $n=2$, with the current configuration,
 $\delta(q_0, a, e) = (q_1, x, y, L, R)$



Theorem 1.2: Every multitape Turing machine has an equivalent single-tape Turing machine

Multidimensional Turing Machines

- A multidimensional TM is one in which the tape can be viewed as extending infinitely in more than one dimension
- The formal definition of a two-dimensional TM involves a transition function δ of the form

$$\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R, U, D\}$$

- where **U** and **D** specify movement of the read-write head up and down, respectively

Nondeterministic Turing Machines

- A nondeterministic TM is like a standard TM, except that δ is now a function

$$\delta: Q \times \Gamma \rightarrow 2^{Q \times \Gamma \times \{L, R\}}$$

- As always when nondeterminism is involved, the range of δ is a set of possible transitions, any of which can be chosen by the machine

Example:

- If a Turing machine has transitions specified by:
 $\delta(q_0, a) = \{(q_1, b, R), (q_2, c, L)\}$, it is nondeterministic. The moves:

$q_0aaa \vdash bq_1aa$

And

$q_0aaa \vdash q_2 \square caa$

are both possible.

Theorem 1.3: The class of deterministic TMs and the class of nondeterministic TMs are equivalent

The Church–Turing thesis

What does computable mean?

Alonzo Church-
Lamda Calculus

- American
Mathematical and
Logician

Allen Turing – **Turing**
Machine
English Computer
Scientist, Mathematician

- **Turning Machine and Lamda Calculus are equivalent in Power.**

**Algorithmically computable means
Computable by a Turing Machine.**

“The notion of an algorithm is equivalent to a TM”

**=>Any thing that can be computed by an algorithm
can be computed by a Turning Machine.**

A Universal Turing Machine

- A standard TM is a special computer
- Once δ is defined, the TM is restricted to carrying out one particular type of computations
- Digital computers, on the other hand, are general purpose machines that can be programmed to do different jobs at different times
- Consequently, TMs cannot be considered equivalent to general purpose digital computers
- This objection can be overcome by designing a reprogrammable TM, called **universal Turing machine**
- A universal TM M_u is an automaton that, given as input the description of any TM M and a string w , can simulate the computation of M on w
- M_u is depicted in Figure 1.3

What is Universal Turing Machine?

- A Universal Turing Machine, is a single machine that can imitate the behavior of an arbitrary Turing machine over any collection of input symbols.
- A single machine that can calculate any computable sequence.
- The UTM would include three pieces of data for the machine it is simulating:
 - ✓ A basic description of the machine.
 - ✓ The contents of the machine tape.
 - ✓ The internal state of the machine
- As a result, a Universal Turing Machine can simulate any other machine.

The schematic diagram of a Universal Turing Machine is as follows:

